

Understanding Linear Regression Through a Single Neuron

From Mathematical Foundations to Python Implementation

Michenchin

August 13, 2026

Contents

1	Before you begin	3
2	Introduction	3
2.1	What is the project?	3
2.2	Project goals	3
3	Linear Regression	4
3.1	What is linear regression?	4
3.2	The meaning of the variables	4
4	The Neuron	4
4.1	How does it work?	4
4.2	Why a single neuron can perform linear regression	4
5	Measuring the error	5
5.1	Loss Function	5
6	The Gradient	5
6.1	Basic Concepts	5
6.1.1	What is a derivative?	5
6.2	Derivative of the loss with respect to w	6
6.3	Derivative of the loss with respect to b	8
7	Gradient Descent	9
8	Complete Cycle	9
9	Personal Conclusion	10
10	A final note	10

1 Before you begin

NOTE

You don't need to understand all the math explained in this document to grasp the basic concepts. Some people might even find it easier to read the code first and then come back to this document. I recommend having both the code and this document open

WARNING

I'm not a teacher, mathematician nor expert in any of the topics covered in this project. I'm just an Spanish 17yo hobbyist trying to learn programming, AI, English, maths and LaTeX by myself that's why I did this project.

2 Introduction

2.1 What is the project?

This project consists of a single-layer perceptron capable of performing linear regression given a dataset of X and Y points, the neuron automatically finds the best-fitting straight line that represents the trend of the data.

To achieve this the neuron utilizes the Gradient Descent optimization algorithm. This method minimizes the prediction error in each iteration by adjusting the neuron internal weight and bias until the model achieves the most accurate linear relationship possible.

2.2 Project goals

This project goal is to understand and implement the math behind machine learning, using its simplest building block: a single neuron. The neuron will be trained to perform linear regression, and, ultimately, to accurately predict the values of y for previously unseen values of x

3 Linear Regression

3.1 What is linear regression?

linear regression is a model that estimates the relationship between a dependent variable and one or more explanatory variables independent variable.

3.2 The meaning of the variables

The neuron has two variables:

1. Weight: w
2. Bias: b

To do the prediction we will use:

$$\hat{y}_i = wx_i + b$$

So w will be the slope of the line, and b will be the y-intercept of the line

4 The Neuron

4.1 How does it work?

The neuron receives an input, then passes it through a function and the output of the function is the prediction.

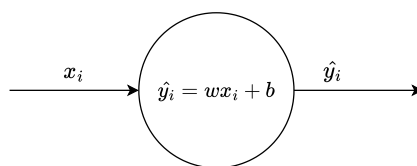


Figure 1: Schematic of how the neuron does a prediction

4.2 Why a single neuron can perform linear regression

A single neuron can do linear regression because as seen in Fig. 1 the internal structure of the neuron is exactly the equation of a straight line.

5 Measuring the error

5.1 Loss Function

A Loss Function is a function we will use for measuring the error in the predictions, we will use the Mean Squared Error (MSE). The MSE is the squared sum of all the errors divided by the number of predictions. Our Loss Function will look like this:

$$L(w, b) = \frac{1}{n} \sum_0^n (wx_i + b - y_i)^2$$

The process is:

$$(1) \text{ } MSE = \sum_0^n (\hat{y}_i - y_i)^2$$

$$(2) \text{ } \hat{y}_i = wx_i + b$$

And if we substitute $\hat{y}_i$ in the MSE function we get:

$$L(w, b) = \frac{1}{n} \sum_0^n (wx_i + b - y_i)^2$$

6 The Gradient

WARNING

This part is the most mathematically complex, since it involves partial derivatives. Despite that, I'll try to explain it as clearly as possible.

6.1 Basic Concepts

6.1.1 What is a derivative?

A derivative is a function in which you enter another function and the result is a new function which describes the exchange rate of the original function in each point.

The mathematical definition of a derivative is this:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

An example with this definition and x^2

$$f(x) = x^2$$

$$f'(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$

If we expand the square binomial:

$$f'(x) = \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$

We factor out the common factor:

$$f'(x) = \lim_{h \rightarrow 0} \frac{h(2x + h)}{h}$$

We canceled the h :

$$f'(x) = \lim_{h \rightarrow 0} \overset{1}{\cancel{h}}(2x + h) \overset{1}{\cancel{h}}$$

Now we have:

$$f'(x) = \lim_{h \rightarrow 0} 2x + h$$

We apply the limit:

$$f'(x) = 2x + 0$$

So:

$$f'(x) = 2x$$

This means that when in $f(x)$ x increases by one unit, its slope increases by 2.

This is a standard derivative. We will use partial derivatives instead. The difference is that a standard derivative considers change in only one direction, whereas partial derivatives can consider changes in multiple directions. But the concepts are the same.

NOTE

Standard derivatives are usually shown as $\frac{dy}{dx}$ and partial derivatives are shown as $\frac{\partial y}{\partial x}$

6.2 Derivative of the loss with respect to w

We need to find $\frac{\partial MSE}{\partial w}$

We know that MSE depends on $\hat{y}_i$ and that $\hat{y}_i$ depends on w

$$(1) \text{ } MSE = \frac{1}{n} \sum_0^n (\hat{y}_i - y_i)^2$$

$$(2) \hat{y}_i = wx_i + b$$

So:

$$MSE \longrightarrow \hat{y}_i \longrightarrow w$$

Knowing this we can say that:

$$\frac{\partial MSE}{\partial w} = \frac{\partial MSE}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial w}$$

NOTE

This is right not because the derivatives act as a fraction and can be canceled, but because of the chain rule. The total effect of changing w is the effect of w on $\hat{y}_i$, multiplied by the effect of $\hat{y}_i$ on the MSE.

Now we have to find $\frac{\partial MSE}{\partial \hat{y}_i}$ and $\frac{\partial \hat{y}_i}{\partial w}$

$$\hat{y}_i = wx_i + b$$

$$\frac{\partial}{\partial w} \hat{y}_i = \frac{\partial}{\partial w} wx_i + b = x_i$$

$$\boxed{\frac{\partial \hat{y}_i}{\partial w} = x_i}$$

Because b is a constant $\frac{\partial}{\partial w} b = 0$ and because We are differentiating with respect to w $\frac{\partial}{\partial w} wx_i = x_i$

Now we are going to find $\frac{\partial MSE}{\partial \hat{y}_i}$

$$\frac{\partial}{\partial \hat{y}_i} MSE = \frac{\partial}{\partial \hat{y}_i} \sum_0^n (\hat{y}_i - y_i)^2$$

$$\frac{\partial}{\partial \hat{y}_i} MSE = 2 \frac{1}{n} \sum_0^n (\hat{y}_i - y_i)$$

$$\boxed{\frac{\partial MSE}{\partial \hat{y}_i} = \frac{2}{n} \sum_0^n (\hat{y}_i - y_i)}$$

Now we can get $\frac{\partial MSE}{\partial w}$

$$\frac{\partial MSE}{\partial w} = \frac{\partial MSE}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial w}$$

$$\frac{\partial MSE}{\partial w} = \frac{2}{n} \sum_0^n (\hat{y}_i - y_i) x_i$$

6.3 Derivative of the loss with respect to b

The process now is very similar as before.

Now we need to find $\frac{\partial MSE}{\partial b}$

We know that MSE depends on $\hat{y}_i$ and that $\hat{y}_i$ depends on b

$$(1) \text{ } MSE = \frac{1}{n} \sum_0^n (\hat{y}_i - y_i)^2$$

$$(2) \text{ } \hat{y}_i = wx_i + b$$

So:

$$MSE \longrightarrow \hat{y}_i \longrightarrow b$$

Knowing this we can say that:

$$\frac{\partial MSE}{\partial b} = \frac{\partial MSE}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial b}$$

Now we only have to find $\frac{\partial \hat{y}_i}{\partial b}$

$$\hat{y}_i = wx_i + b$$

$$\frac{\partial}{\partial b} \hat{y}_i = \frac{\partial}{\partial b} wx_i + b$$

$$\frac{\partial \hat{y}_i}{\partial b} = 1$$

Because $w x_i$ are constants $\frac{\partial}{\partial b} w x_i = 0$ and because we are differentiating with

respect to b $\frac{\partial}{\partial b} b = 1$

Now we can get $\frac{\partial MSE}{\partial b}$

$$\frac{\partial MSE}{\partial b} = \frac{\partial MSE}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial b}$$

$$\frac{\partial MSE}{\partial b} = \frac{2}{n} \sum_0^n (\hat{y}_i - y_i)$$

7 Gradient Descent

Now that we have $\frac{\partial MSE}{\partial w}$ and $\frac{\partial MSE}{\partial b}$ we will use that to adjust the predictions by adjusting the weight and bias.

So after we do a prediction we adjust the weight and bias like this:

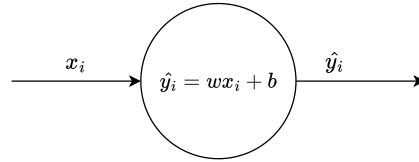
$$w \leftarrow w - \eta \frac{\partial MSE}{\partial w}$$
$$b \leftarrow b - \eta \frac{\partial MSE}{\partial b}$$

Where η is the learning rate.

The **negative is important** because we want to move in the opposite direction of the gradient, towards a lower MSE.

8 Complete Cycle

1. The neuron receives a list of points x
2. Each point x_i passes through the function $\hat{y}_i = wx_i + b$ and each point $\hat{y}_i$ is stored on the list of predictions



3. Given the prediction points $\hat{y}$ and the real points y we calculate the MSE:

$$MSE = \frac{1}{n} \sum_0^n (\hat{y}_i - y_i)^2$$

4. Using the gradients $\frac{\partial MSE}{\partial w}$ and $\frac{\partial MSE}{\partial b}$ we adjust the weight and bias:

$$w \leftarrow w - \eta \frac{\partial MSE}{\partial w}$$
$$b \leftarrow b - \eta \frac{\partial MSE}{\partial b}$$

5. Repeat until the error is as lowest as it can be.

$x \longrightarrow \hat{y} \longrightarrow MSE \longrightarrow Gradients \longrightarrow w, b \text{ update} \longrightarrow Repeat$
--

9 Personal Conclusion

This project helped me understand what is happening behind a simple neural network instead of just using it as a black box. I learned how predictions, loss functions, derivatives and Gradient Descent work together to make the neuron learn.

This is only a small example, but it provides the mathematical foundations needed to understand more complex neural networks.

10 A final note

If you are reading this, I hope I explained everything clearly and that this document has helped you better understand how a neural network works.

Thank you for reading! :)